



UNIVERSIDAD DE LAS FUERZAS ARMADAS-ESPE
DEPARTAMENTO DE CIENCIAS DE LA COMPUTACIÓN - DCCO-SS
CARRERA DE INGENIERÍA EN TECNOLOGÍAS DE LA INFORMACIÓN



PERIODO	:	PREGRADO ABRIL 2026-AGOSTO 2026
ASIGNATURA	:	Fundamentos de sistemas web
TEMA	:	Taller de interacciones web
ESTUDIANTE	:	Angel Steven Rodriguez Chavez
NIVEL-PARALELO - NRC	:	29535
DOCENTE	:	Ing. Geovanny José Brito Casanova
FECHA DE ENTREGA	:	17/07/2026
ENLACE DE GITHUB	:	https://github.com/angeldev7/P3-Taller

SANTO DOMINGO – ECUADOR

Tabla de contenido

1.	Introducción.....	3
2.	Objetivos.....	3
2.1.	Objetivo General:	3
2.2.	Objetivos Específicos:	4
3.	Desarrollo / Marco Teórico/ Práctica.....	4
3.1.	Marco conceptual breve.....	4
3.2.	Desarrollo Paso a Paso de los Ejercicios	5
3.3.	Explicación de código relevante.....	9
3.4.	Simulación de Capturas de Pantalla (Pruebas Realizadas).....	12
3.5.	Análisis de resultados	15
4.	Conclusiones.....	16
5.	Recomendaciones	17
6.	Criterio personal	17
7.	Bibliografía/ Referencias	18

Tabla de figuras

Figure 1	Formulario académico con la cuarta nota agregada	12
Figure 2	Validación de campos obligatorios y nombres con caracteres especiales	13
Figure 3	Resultado de la evaluación con beca calculada según carrera y promedio	13
Figure 4	Representación JSON del historial completo de estudiantes	14
Figure 5	Tabla del historial de estudiantes evaluados con .map()	14
Figure 6	Dashboard con estadísticas generales del curso	14
Figure 7	Búsqueda de estudiante por nombre con filter() e includes()	15
Figure 8	Ranking de estudiantes por promedio con sort().....	15

Tabla de código

Código 1	Renderizado de la tabla con .map()	10
Código 2	Lógica del promedio especial	10
Código 3	Búsqueda parcial filter() e includes().....	11
Código 4	Ranking con sort()	11
Código 5	Validación de Nombres Repetidos con some()	11

1. Introducción

En el desarrollo de aplicaciones web interactivas, JavaScript juega un papel fundamental al permitir la manipulación dinámica del DOM, la gestión de eventos, el procesamiento de datos y la creación de experiencias de usuario fluidas. El presente taller tiene como objetivo ampliar un evaluador académico previamente desarrollado en clase, incorporando nuevas funcionalidades que refuercen los conceptos de funciones, eventos, validaciones, arreglos, objetos, estructuras condicionales, ciclos y manipulación del DOM.

La aplicación base consiste en un formulario que permite ingresar datos de un estudiante (nombre, carrera y tres notas), calcular su promedio, clasificar su estado académico y mostrar los resultados en pantalla. A partir de esta base, se han implementado mejoras como la adición de una cuarta nota, cálculo de promedio especial sin la nota más baja, generación de recomendaciones académicas, clasificación de rendimiento, historial de estudiantes evaluados, búsqueda y ranking, y un dashboard con estadísticas generales del curso.

Todas las funcionalidades han sido desarrolladas utilizando JavaScript vanilla, Bootstrap 5 para los estilos y componentes visuales, y siguiendo buenas prácticas de programación. El resultado final es una aplicación web completa y funcional que demuestra la aplicación de los conceptos fundamentales de interacción web con JavaScript.

2. Objetivos

2.1. Objetivo General:

Implementar un sistema interactivo de evaluación y registro académico utilizando JavaScript nativo para el procesamiento de datos del lado del cliente y Bootstrap para el

diseño visual responsivo, aplicando estructuras de datos dinámicas y métodos de manipulación del DOM que resuelvan problemas lógicos y estadísticos del entorno universitario.

2.2. Objetivos Específicos:

1. Validar y procesar datos numéricos decimales en un formulario de cuatro notas, controlando rangos de valores entre 0 y 20 de manera estricta.
2. Desarrollar lógica condicional anidada para la clasificación de rendimiento, asignación automatizada de becas estudiantiles en base a carrera y promedio, y generación de recomendaciones académicas.
3. Administrar arreglos dinámicos de objetos en memoria global para almacenar el historial de evaluaciones, impidiendo duplicados y permitiendo búsquedas de texto.
4. Diseñar un dashboard estadístico integrando acumuladores, comparadores y filtros que calculen promedios globales y localicen alumnos con métricas extremas (mejor y peor promedio).
5. Implementar funcionalidades de búsqueda y ranking para facilitar la gestión y visualización del historial de estudiantes.

3. Desarrollo / Marco Teórico/ Práctica

3.1. Marco conceptual breve

- **Manipulación del DOM:** Es la capacidad de JavaScript de acceder y modificar dinámicamente la estructura, estilo y contenido del documento

HTML utilizando la API del navegador a través de métodos como `document.getElementById()`.

- **Métodos de Arreglos Modernos:**

- `map()`: Transforma cada elemento de un arreglo y devuelve un nuevo arreglo. Se utiliza para generar las filas de la tabla de historial.
- `filter()`: Crea un nuevo arreglo con los elementos que cumplen una condición. Se usa para la búsqueda de estudiantes.
- `some()`: Verifica si al menos un elemento cumple una condición. Se usa para validar nombres repetidos.
- `sort()`: Ordena los elementos de un arreglo. Se usa para generar el ranking de promedios.

- **Estructuras de Control y Validación:** El uso de condicionales (`if-else`, `switch`) y bucles (`for...of`) permite evaluar si las notas y nombres cumplen con los parámetros predefinidos antes de guardarlos.

- **Bootstrap 5:** Framework de CSS utilizado para maquetar el sistema con tarjetas (`card`), alertas (`alert`), tablas adaptativas (`table-responsive`) y botones interactivos sin requerir estilos CSS adicionales.

3.2. Desarrollo Paso a Paso de los Ejercicios

A continuación, se detalla cómo se resolvieron cada uno de los 20 requerimientos de la actividad:

1. Agregar una cuarta nota al formulario

Se añadió un input con id nota4 en index.html y se modificó app.js para leer su valor mediante `document.getElementById("nota4").value`. El arreglo de notas ahora contiene cuatro elementos.

2. Permitir notas con decimales

Se usó `parseFloat()` para las cuatro entradas. En los resultados visuales de los promedios se utilizó el método `.toFixed(2)` para limitar la salida a dos decimales. Además, los inputs tienen `step="0.01"` para permitir valores decimales.

3. Ajustar las carreras del formulario

Se editó el componente `<select id="carrera">` en index.html para incluir las opciones exactas: Tecnologías de la Información (TI), Software y Sistemas, reemplazando la opción base.

4. Crear mensaje de beca académica

Se estructuró la función `calcularBeca()` aplicando las reglas solicitadas:

TI y promedio > 18 = Beca 100%.

Software y promedio > 17 = Beca 80%.

Sistemas y promedio > 16 = Beca 60%.

De lo contrario = No aplica.

5. Mostrar colores según el tipo de beca

Conectando con Bootstrap, se asoció cada beca a clases oficiales de alerta: `alert-success` (100%), `alert-primary` (80%), `alert-warning` (60%) y `alert-secondary` (sin beca).

6. Calcular promedio especial sin la nota más baja

Se programó `calcularPromedioEspecial(notas)` usando `Math.min(...notas)` para hallar la nota menor. Se utiliza una bandera booleana para descartar únicamente la primera ocurrencia de la nota más baja, calculando el promedio de las tres notas restantes.

7. Mostrar nota más alta y nota más baja

Se implementaron las funciones `obtenerNotaAlta(notas)` y `obtenerNotaBaja(notas)` invocando `Math.max(...notas)` y `Math.min(...notas)`.

8. Contar notas aprobadas, en recuperación y reprobadas

La función `contarNotasPorEstado(notas)` evalúa individualmente cada nota con un ciclo `for...of`, asignando contadores según: Aprobadas (≥ 14), Recuperación (≥ 10 y < 14) o Reprobadas (< 10).

9. Generar recomendación académica

Se programó una función que evalúa el promedio y asigna un texto instructivo personalizado:

18 a 20: Mantener desempeño y apoyar a compañeros.

14 a 17.99: Reforzar temas específicos.

10 a 13.99: Asistir a tutorías y practicar ejercicios.

0 a 9.99: Repetir contenidos base y solicitar acompañamiento.

10. Clasificar el rendimiento académico

Se creó una escala cualitativa (Alto, Medio, Básico y Bajo) basada en el promedio final, asignando colores dinámicos con clases de Bootstrap.

11. Crear historial de estudiantes evaluados

Se declaró un array global `let estudiantes = []`; que almacena los objetos de los alumnos que pasan exitosamente las validaciones.

12. Validar nombres repetidos

Antes de añadir un nuevo estudiante, el método `estudiantes.some()` verifica si el nombre ingresado ya existe (ignorando mayúsculas/minúsculas). Si es duplicado, lanza una alerta y aborta la operación.

13. Crear botón para listar estudiantes

Se insertó un botón `btnListar` y la tabla dinámica que lee el historial completo. Las filas se generan mediante el método `map()`, que transforma cada objeto estudiante en una cadena HTML y las une con `join("")`.

14. Mostrar cuántos estudiantes han sido evaluados

El dashboard cuenta con un componente que lee `estudiantes.length` y lo actualiza en pantalla mediante `actualizarEstadisticasCurso()`.

15. Contar aprobados, recuperación y reprobados del curso

Se recorre el historial completo con un ciclo `for...of` sumando los estados generales de los estudiantes para presentarlos en el dashboard.

16. Calcular promedio general del curso

Suma los promedios individuales acumulados en el historial y los divide entre el total de estudiantes evaluados.

17. Buscar estudiante por nombre

Se añadió un input de texto txtBuscar y un botón que filtra el arreglo de estudiantes usando filter() y includes() (con conversión a minúsculas para búsqueda insensible a mayúsculas), y vuelve a renderizar la tabla con las coincidencias.

18. Crear ranking de estudiantes

El botón "Ranking de Promedios" clona el arreglo general con el operador spread [...estudiantes], lo ordena descendientemente con sort((a, b) => b.promedio - a.promedio) y actualiza la tabla.

19. Mostrar estudiante con mayor y menor promedio

Dos tarjetas informativas dinámicas muestran el nombre y el promedio exacto de los estudiantes con mejor y peor desempeño del curso, identificados mediante comparaciones durante el recorrido del historial.

20. Mejorar el objeto final del estudiante

El objeto final cuenta con todos los atributos requeridos (nombre, carrera, notas, promedio, promedioEspecial, notaMayor, notaMenor, estado, rendimiento, beca y recomendación). Se presenta en formato JSON mostrando el historial completo en la sección correspondiente..

3.3. Explicación de código relevante

```
1. function listarEstudiantes(lista) {  
2.     const tbody = document.getElementById("tablaHistorial");  
3.  
4.     if (lista.length === 0) {
```

```

5.         tbody.innerHTML = `<tr><td colspan="10" class="text-muted py-4">No hay estudiantes
registrados.</td></tr>`;
6.         return;
7.     }
8.
9.     tbody.innerHTML = lista.map(est => `
10.         <tr>
11.             <td class="fw-bold">${est.nombre}</td>
12.             <td>${est.carrera}</td>
13.             <td>[${est.notas.join(", ")}]</td>
14.             <td class="fw-bold">${est.promedio.toFixed(2)}</td>
15.             <td class="text-success fw-bold">${est.promedioEspecial.toFixed(2)}</td>
16.             <td>${est.notaMayor.toFixed(2)}</td>
17.             <td class="text-danger">${est.notaMenor.toFixed(2)}</td>
18.             <td><span class="badge bg-
${obtenerColorEstado(est.estado)}">${est.estado}</span></td>
19.             <td>
20.                 <span class="badge bg-${est.rendimiento === "Alto" ? "success" :
est.rendimiento === "Medio" ? "primary" : est.rendimiento === "Básico" ? "warning text-dark" :
"danger"}">
21.                     ${est.rendimiento}
22.                 </span>
23.             </td>
24.             <td><small class="fw-semibold">${est.beca}</small></td>
25.         </tr>
26.     `).join("");
27. }
28.

```

Código 1 Renderizado de la tabla con .map()

Explicación: Esta función utiliza map() para transformar cada objeto estudiante en una fila de tabla HTML. join("") convierte el arreglo resultante en una sola cadena. Si la lista está vacía, muestra un mensaje informativo. El operador ternario asigna colores dinámicos a las insignias de rendimiento.

```

1. function calcularPromedioEspecial(notas) {
2.     const minNota = Math.min(...notas);
3.     let descartado = false;
4.     const notasFiltradas = notas.filter(nota => {
5.         if (nota === minNota && !descartado) {
6.             descartado = true;
7.             return false;
8.         }
9.         return true;
10.    });
11.    const suma = notasFiltradas.reduce((acc, curr) => acc + curr, 0);
12.    return suma / notasFiltradas.length;
13. }
14.

```

Código 2 Lógica del promedio especial

Explicación: Esta función identifica la nota más baja con Math.min. Para evitar descartar notas repetidas idénticas de forma errónea (por ejemplo, si un estudiante tiene dos

notas de 12 y ambas son las más bajas), se utiliza una bandera booleana descartado que asegura eliminar únicamente una sola instancia de la nota menor.

```
1. function buscarEstudiante() {
2.     const query = document.getElementById("txtBuscar").value.trim().toLowerCase();
3.     if (query === "") {
4.         alert("Por favor ingrese un término de búsqueda.");
5.         return;
6.     }
7.     const filtrados = estudiantes.filter(est => est.nombre.toLowerCase().includes(query));
8.     listarEstudiantes(filtrados);
9. }
10.
```

Código 3 Búsqueda parcial filter() e includes()

Explicación: Convierte el término de búsqueda ingresado y los nombres registrados a minúsculas (toLowerCase), permitiendo búsquedas insensibles a mayúsculas. filter() devuelve un nuevo arreglo con las coincidencias que contienen la cadena buscada mediante includes().

```
1. function generarRanking() {
2.     if (estudiantes.length === 0) {
3.         alert("No hay estudiantes en el historial para ordenar.");
4.         return;
5.     }
6.     const ranking = [...estudiantes].sort((a, b) => b.promedio - a.promedio);
7.     listarEstudiantes(ranking);
8. }
9.
```

Código 4 Ranking con sort()

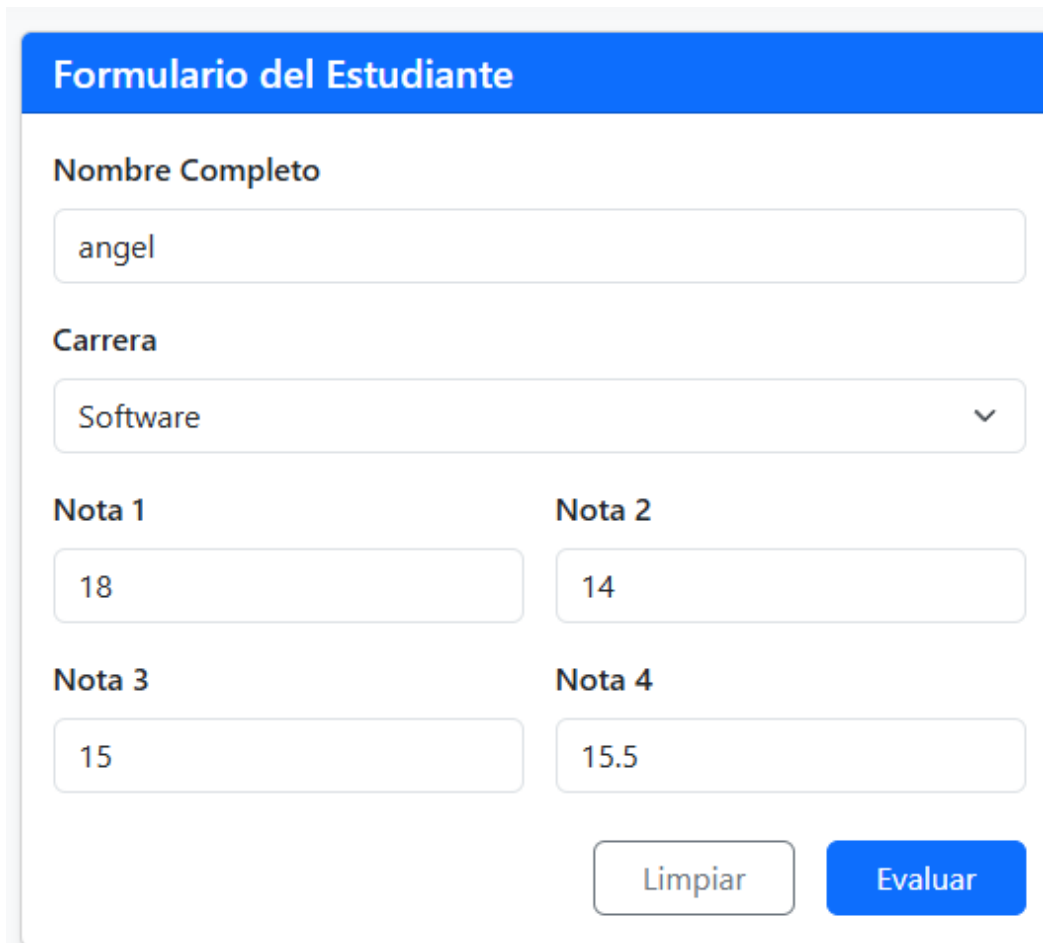
Explicación: Clona el arreglo original con el operador spread [...estudiantes] para no modificar el historial original. sort() lo ordena de mayor a menor promedio restando a.promedio - b.promedio (orden ascendente) o b.promedio - a.promedio (descendente). Finalmente, se muestra el resultado en la tabla.

```
1. const existeEstudiante = estudiantes.some(est => est.nombre.toLowerCase() ===
nombre.toLowerCase());
2. if (existeEstudiante) {
3.     alert(`Error: Ya existe un estudiante registrado con el nombre "${nombre}".`);
4.     return;
5. }
6.
```

Código 5 Validación de Nombres Repetidos con some()

Explicación: `some()` recorre el arreglo y retorna true si encuentra al menos un elemento que cumpla la condición. Esto permite verificar de manera eficiente si el nombre ingresado ya existe en el historial.

3.4. Simulación de Capturas de Pantalla (Pruebas Realizadas)



The screenshot shows a web form titled "Formulario del Estudiante" with a blue header. The form contains the following fields and controls:

- Nombre Completo:** A text input field containing the value "angel".
- Carrera:** A dropdown menu with "Software" selected and a downward arrow icon.
- Nota 1:** A text input field containing the value "18".
- Nota 2:** A text input field containing the value "14".
- Nota 3:** A text input field containing the value "15".
- Nota 4:** A text input field containing the value "15.5".
- Buttons:** Two buttons at the bottom right: "Limpiar" (white with a gray border) and "Evaluar" (solid blue).

Figure 1 Formulario académico con la cuarta nota agregada

127.0.0.1:5500 dice

El nombre del estudiante debe tener al menos 5 caracteres.

Aceptar

Formulario del Estudiante

Nombre Completo

123

Ingrese los datos en el formulario y

127.0.0.1:5500 dice

El nombre solo debe contener letras y espacios.

Aceptar

Formulario del Estudiante

Nombre Completo

123 345

Ingrese los datos en el formulario y

Figure 2 Validación de campos obligatorios y nombres con caracteres especiales

Resultado de la Evaluación

Estudiante: angel, carrera: Software, promedio: 17.25, estado: Aprobado, Nota Más Alta: 19.00, Nota Más Baja: 15.00, Aprobadas: 4, Recuperación: 0, Reprobadas: 0

Beca: Aplica para beca del 80%

Promedio Normal
17.25

Prom. Especial
18.00

Nota Máxima
19.00

Nota Mínima
15.00

Rendimiento Académico:
MEDIO

Recomendación:
14 a 17.99: Reforzar temas específicos.

AP: 4 | REC: 0 | REP: 0

Figure 3 Resultado de la evaluación con beca calculada según carrera y promedio



Figure 4 Representación JSON del historial completo de estudiantes

Historial General de Alumnos										Ranking de Promedios	Listar Todos
Buscar estudiante por nombre...										Buscar	Restaurar
Estudiante	Carrera	Notas	Promedio	Prom. Especial	Nota Alta	Nota Baja	Estado	Rendimiento	Beca		
angel	Software	[15, 18, 17, 19]	17.25	18.00	19.00	15.00	Aprobado	Medio	Aplica para beca del 80%		
Steven	TI	[17, 14, 15, 13]	14.75	15.33	17.00	13.00	Aprobado	Medio	No aplica para beca		
Juan Carlos	Sistemas	[18, 14, 15, 16]	15.75	16.33	18.00	14.00	Aprobado	Medio	No aplica para beca		

Figure 5 Tabla del historial de estudiantes evaluados con .map()



Historial General de Alumnos

Ranking de Promedios

Listar Todos

angel

BuscarRestaurar

Estudiante	Carrera	Notas	Promedio	Prom. Especial	Nota Alta	Nota Baja	Estado	Rendimiento	Beca
angel	Software	[15, 18, 17, 19]	17.25	18.00	19.00	15.00	Aprobado	Medio	Aplica para beca del 80%

Figure 7 Búsqueda de estudiante por nombre con `filter()` e `includes()`

Historial General de Alumnos							Ranking de Promedios	Listar Todos	
Buscar estudiante por nombre... Buscar Restaurar									
Estudiante	Carrera	Notas	Promedio	Prom. Especial	Nota Alta	Nota Baja	Estado	Rendimiento	Beca
angel	Software	[15, 18, 17, 19]	17.25	18.00	19.00	15.00	Aprobado	Medio	Aplica para beca del 80%
Juan Carlos	Sistemas	[18, 14, 15, 16]	15.75	16.33	18.00	14.00	Aprobado	Medio	No aplica para beca
Steven	TI	[17, 14, 15, 13]	14.75	15.33	17.00	13.00	Aprobado	Medio	No aplica para beca

Figure 8 Ranking de estudiantes por promedio con `sort()`

3.5. Analisis de resultados

Las pruebas ejecutadas demuestran que el sistema procesa con exactitud los datos numéricos y de texto:

1. Validaciones: Las validaciones impiden el ingreso de caracteres inválidos (usando `regexNombre`), notas fuera del rango `[0, 20]`, nombres menores a 5 caracteres y nombres duplicados, garantizando la consistencia de la información académica.
2. Cálculos: La lógica de promedios se ejecuta perfectamente; el promedio especial elimina de manera efectiva solo una ocurrencia del número más bajo (gracias a la bandera `descartado`), recalculando sobre las 3 notas restantes.
3. Métodos modernos de arreglos: El uso de `map()` para renderizar la tabla, `filter()` para búsquedas, `some()` para validación de duplicados y `sort()` para ranking demuestra la aplicación de métodos funcionales que hacen el código más legible y eficiente.

4. Reactividad: El motor de búsqueda y el ranking operan de forma reactiva sin recargar la página, actualizando solo la tabla mediante manipulación del DOM, lo que proporciona una experiencia de usuario rápida y dinámica.
5. Dashboard: El dashboard se actualiza automáticamente cada vez que se agrega un nuevo estudiante, reflejando los cambios en tiempo real en los contadores de evaluados, promedio del curso, estados y mejores/peores promedios.

4. Conclusiones

1. La manipulación del DOM mediante JavaScript nativo permitió crear páginas dinámicas altamente interactivas, reduciendo la necesidad de depender de librerías externas y optimizando la velocidad de respuesta en el navegador.
2. El uso de arreglos globales de objetos facilitó el almacenamiento temporal de datos estructurados, permitiendo realizar operaciones complejas de ordenamiento (ranking) y filtrado (búsqueda) del lado del cliente en fracciones de segundo.
3. La integración estructurada de clases utilitarias de Bootstrap permitió estilizar interfaces complejas de manera estandarizada y responsiva, logrando una presentación profesional apta para la visualización de dashboards de datos.
4. Las validaciones implementadas (campos obligatorios, rango de notas, longitud de nombre, nombres repetidos, caracteres especiales) garantizan la integridad de los datos y previenen errores en el procesamiento.

5. La implementación de búsqueda, ranking, promedio especial y dashboard demuestra la capacidad de resolver problemas lógicos complejos utilizando estructuras de control, funciones y métodos de arreglos.

5. Recomendaciones

1. Se recomienda implementar almacenamiento persistente del lado del cliente como localStorage o una base de datos externa para evitar que el historial de estudiantes y las estadísticas del dashboard se pierdan al recargar el navegador.
2. Es aconsejable modularizar las validaciones complejas de entrada en funciones independientes para facilitar el mantenimiento futuro del código y permitir la reutilización de reglas de negocio en otros formularios.
3. Se sugiere añadir efectos de transición visual o micro-animaciones en las alertas del DOM para mejorar la experiencia visual del usuario cuando se agregan nuevos registros o se realizan búsquedas en la tabla.

6. Criterio personal

¿Qué aprendí?

Durante el desarrollo de este taller aprendí a conectar de manera directa los flujos lógicos de JavaScript con interfaces de usuario maquetadas en HTML. Comprendí cómo gestionar estados dinámicos en memoria utilizando arreglos de objetos complejos, y cómo utilizar métodos avanzados del prototipo de arreglos como `.some()`, `.filter()`, y `.sort()` para resolver requerimientos de negocio de forma limpia.

¿Qué parte me resultó más compleja?

La parte que me resultó más compleja fue diseñar el algoritmo de promedio especial para descartar únicamente la primera ocurrencia de la nota más baja. Si un estudiante tenía notas repetidas que coincidían con la mínima (por ejemplo: [12, 12, 14, 15]), un simple `.filter()` que removiera el valor mínimo eliminaría ambas notas 12. Tuve que idear una lógica basada en banderas booleanas (`descartado = true`) para asegurarme de que solo se excluyera un valor, manteniendo las repeticiones correctas para la estadística.

¿Qué funcionalidad considero más útil?

Considero que la funcionalidad más útil es el Dashboard Estadístico del Curso y el Ranking. En un entorno académico real, tener visibilidad instantánea del promedio general, del número de aprobados y de los estudiantes con mejor/peor rendimiento ayuda a tomar decisiones rápidas de tutoría y seguimiento académico.

¿Cómo me ayudó este taller?

Este taller me dio una visión clara de cómo se estructuran las aplicaciones SPA (Single Page Application) básicas. Me permitió entender que la interactividad no es solo mostrar mensajes en alertas del navegador, sino inyectar componentes enriquecidos, tablas y dashboards en tiempo real modificando el árbol de nodos de la página.

7. Bibliografía/ Referencias

- MDN Web Docs. (2026). *Document Object Model (DOM)*. Mozilla Developer Network. https://developer.mozilla.org/es/docs/Web/API/Document_Object_Model. Fecha de consulta: 17 de julio de 2026.
- MDN Web Docs. (2026). *JavaScript Guide*. Mozilla Corporation. <https://developer.mozilla.org/es/docs/Web/JavaScript/Guide>. Fecha de consulta: 17 de julio de 2026.

- Bootstrap. (2026). *Introduction to Bootstrap v5.3*.
GetBootstrap. <https://getbootstrap.com/docs/5.3/getting-started/introduction/>. Fecha de consulta: 17 de julio de 2026.